



编译原理

第七章 语义分析和中间代码产生

丁志军

dingzj@tongji.edu.cn



■ 中间语言（复杂性介于源语言和目标语言之间）的好处：

- 便于进行与机器无关的代码优化工作
- 易于移植
- 使编译程序的结构在逻辑上更为简单明确

内容线索

1. 中间语言

2. 说明语句的翻译

3. 赋值语句的翻译

4. 布尔表达式的翻译

5. 两遍扫描条件控制及其语句的翻译

6. 一遍扫描条件控制及其语句的翻译

7. 过程调用的处理

■ 常用的中间语言

- 后缀式，逆波兰表示
- 图表示
 - DAG
 - 抽象语法树
- 三地址代码
 - 三元式
 - 四元式
 - 间接三元式

后缀式

- **后缀式**表示法：Lukasiewicz发明的一种表示表达式的方法，又称**逆波兰**表示法。
- 一个表达式E的后缀形式可以如下定义：
 1. 如果E是一个变量或常量，则E的后缀式是E自身。
 2. 如果E是 $E_1 \text{ op } E_2$ 形式的表达式，其中op是任何二元操作符，则E的**后缀式**为 $E_1' E_2' \text{ op}$ ，其中 E_1' 和 E_2' 分别为 E_1 和 E_2 的后缀式。
 3. 如果E是 (E_1) 形式的表达式，则 E_1 的后缀式就是E的后缀式。

- **逆波兰表示法不用括号。只要知道每个算符的目数，对于后缀式，不论从哪一端进行扫描，都能对它进行唯一分解。**
- **后缀式的计算**
 - **用一个栈实现。**
 - **一般的计算过程是：自左至右扫描后缀式，每碰到运算符就把它推进栈。每碰到 k 目运算符就把它作用于栈顶的 k 个项，并用运算结果代替这 k 个项。**

把表达式翻译成后缀式的语义规则描述

产生式	语义规则
$E \rightarrow E^{(1)} \text{ op } E^{(2)}$	$E.\text{code} := E^{(1)}.\text{code} E^{(2)}.\text{code} \text{op}$
$E \rightarrow (E^{(1)})$	$E.\text{code} := E^{(1)}.\text{code}$
$E \rightarrow \text{id}$	$E.\text{code} := \text{id}$

- $E.\text{code}$ 表示E后缀形式
- op 表示任意二元操作符
- “||” 表示后缀形式的连接

$E \rightarrow E^{(1)} \text{op } E^{(2)}$

$E.\text{code} := E^{(1)}.\text{code} \parallel E^{(2)}.\text{code} \parallel \text{op}$

$E \rightarrow (E^{(1)})$

$E.\text{code} := E^{(1)}.\text{code}$

$E \rightarrow \text{id}$

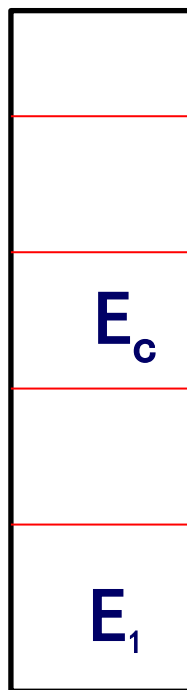
$E.\text{code} := \text{id}$

- 数组POST存放后缀式：k为下标，初值为1
- 上述语义动作可实现为：

产生式	程序段
$E \rightarrow E^{(1)} \text{op } E^{(2)}$	{POST[k]:=op;k:=k+1} print op
$E \rightarrow (E^{(1)})$	{ }
$E \rightarrow i$	{POST[k]:=i;k:=k+1} print i

$a+b+c$ 后綴式的生成

$E \rightarrow E^{(1)} \text{op } E^{(2)}$ { POST[k] := op; k:=k+1 }
print op
 $E \rightarrow (E^{(1)})$ { }
 $E \rightarrow i$ { POST[k] := i; k:=k+1 }
{print i}



输入

$a + b + c$

输出

a	b	+	c	+	
---	---	---	---	---	--

用 $E \rightarrow E \text{ op } E$ 归约, print +

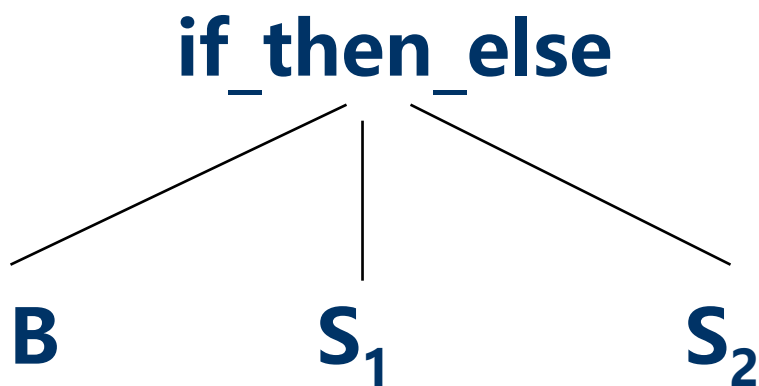
图表示法

- 抽DAG
- 象语法树

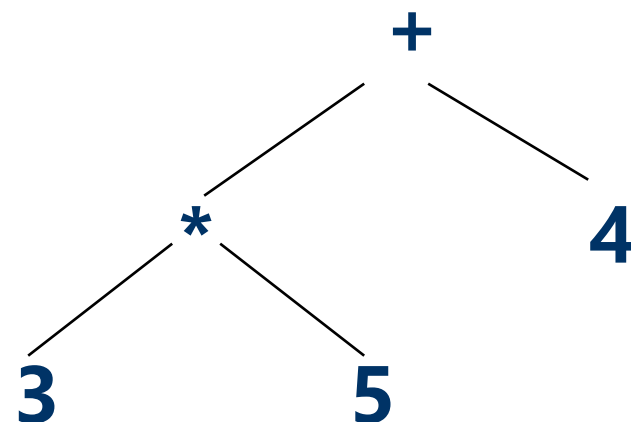
抽象语法树

- 在语法树中去掉那些对翻译不必要的信息，从而获得更有效的源程序中间表示。这种经变换后的语法树称之为**抽象语法树**(Abstract Syntax Tree)

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



$3 * 5 + 4$



建立表达式的抽象语法树

- **mknode (op,left,right)** 建立一个运算符结点，标号是op，两个域left和right分别指向左子树和右子树。
- **mkleaf (id,entry)** 建立一个标识符结点，标号为id，一个域entry指向标识符在符号表中的入口。
- **mkleaf (num,val)** 建立一个数结点，标号为num，一个域val用于存放数的值。

建立抽象语法树的语义规则

产生式

$E \rightarrow E_1 + T$

$E \rightarrow E_1 - T$

$E \rightarrow T$

$T \rightarrow (E)$

$T \rightarrow \text{id}$

$T \rightarrow \text{num}$

语义规则

$E.\text{nptr} := \text{mknode} ('+' , E_1.\text{nptr}, T.\text{nptr})$

$E.\text{nptr} := \text{mknode} ('-' , E_1.\text{nptr}, T.\text{nptr})$

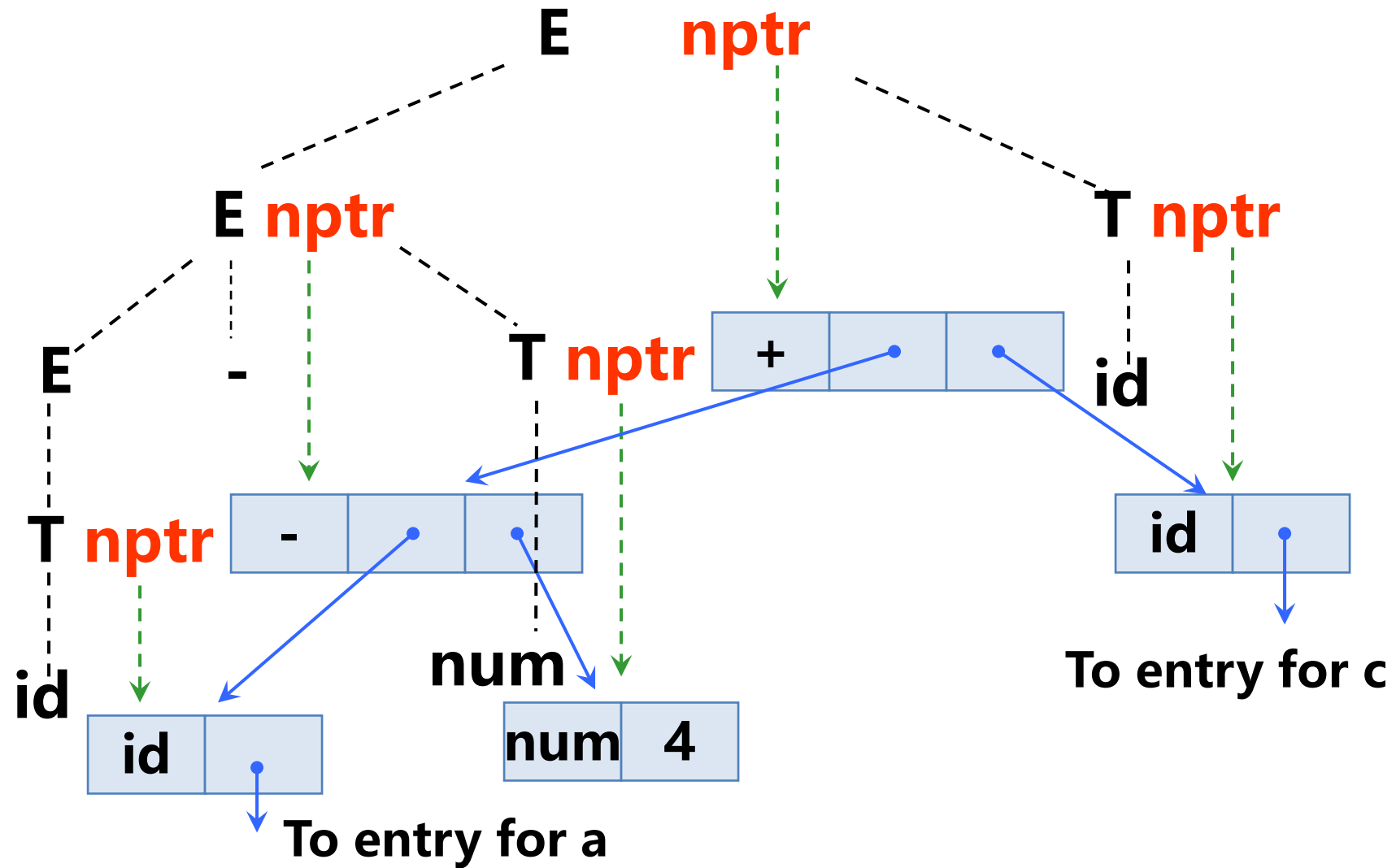
$E.\text{nptr} := T.\text{nptr}$

$T.\text{nptr} := E.\text{nptr}$

$T.\text{nptr} := \text{mkleaf} (\text{id}, \text{id.entry})$

$T.\text{nptr} := \text{mkleaf} (\text{num}, \text{num.val})$

a - 4 + c 的抽象语法树



产生赋值语句抽象语法树的属性文法

产生式

$S \rightarrow \text{id} := E$

$E \rightarrow E_1 + E_2$

$E \rightarrow E_1 * E_2$

$E \rightarrow -E_1$

$E \rightarrow (E_1)$

$E \rightarrow \text{id}$

语义规则

$S.\text{nptr} := \text{mknode}(\text{'assign'}, \text{mkleaf}(\text{id}, \text{id.place}), E.\text{nptr})$

$E.\text{nptr} := \text{mknode}(\text{'+'}, E_1.\text{nptr}, E_2.\text{nptr})$

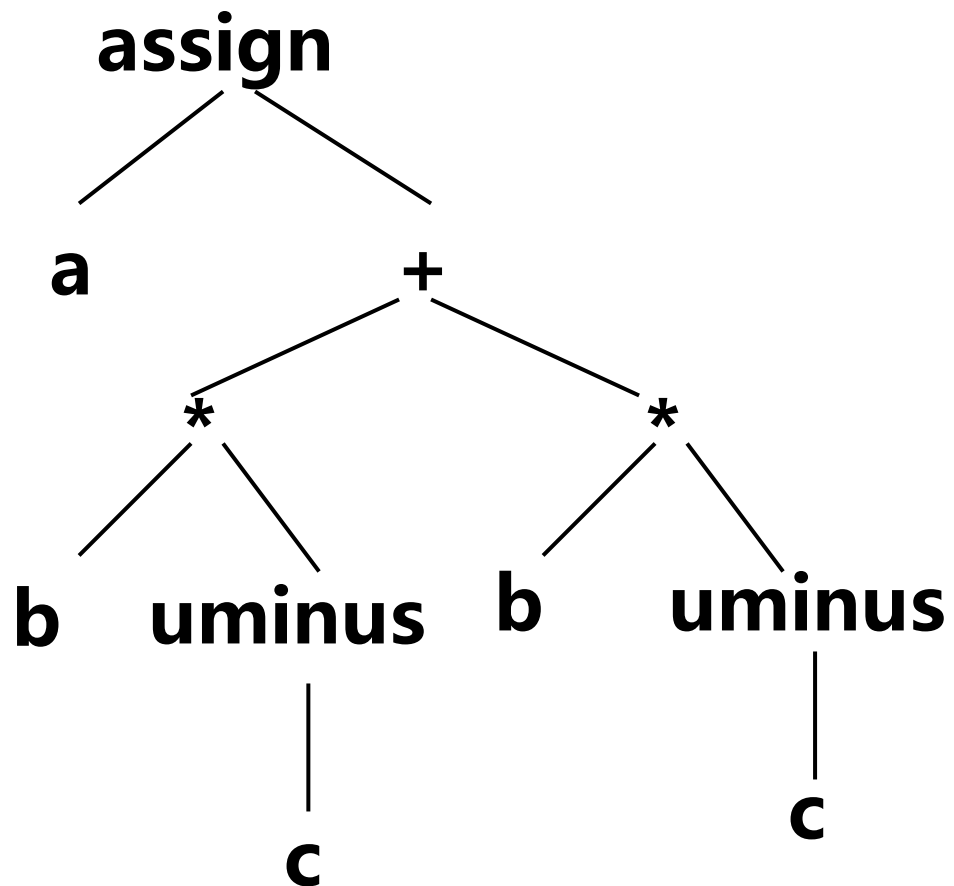
$E.\text{nptr} := \text{mknode}(\text{'*'}, E_1.\text{nptr}, E_2.\text{nptr})$

$E.\text{nptr} := \text{mknode}(\text{'uminus'}, E_1.\text{nptr})$

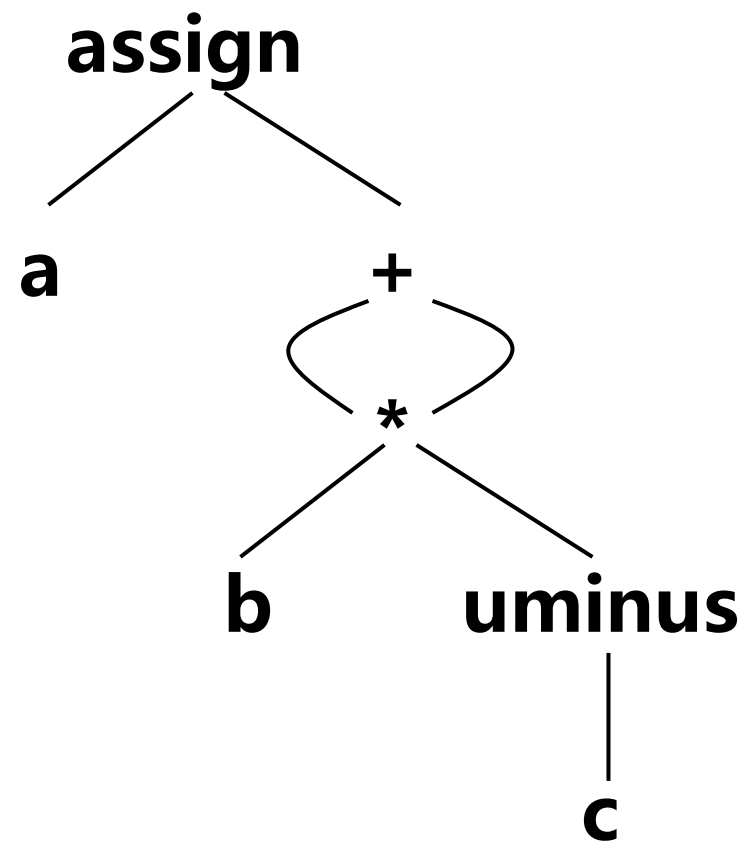
$E.\text{nptr} := E_1.\text{nptr}$

$E.\text{nptr} := \text{mkleaf}(\text{id}, \text{id.place})$

$a := b * (-c) + b * (-c)$ 的图表示法



抽象语法树



DAG

- **有向无循环图(Directed Acyclic Graph, 简称DAG)**
 - 对表达式中的每个子表达式, DAG中都有一个结点
 - 一个**内部结点**代表一个**操作符**, 它的孩子代表操作数
 - 在一个DAG中代表公共子表达式的结点具有多个父结点

$a + a * (b - c) + (b - c) * d$ 的图表示法

■ 后缀表达式: $aabc-*+bc-d*+$

产生式

$E \rightarrow E_1 + T$

$E \rightarrow E_1 - T$

$E \rightarrow T$

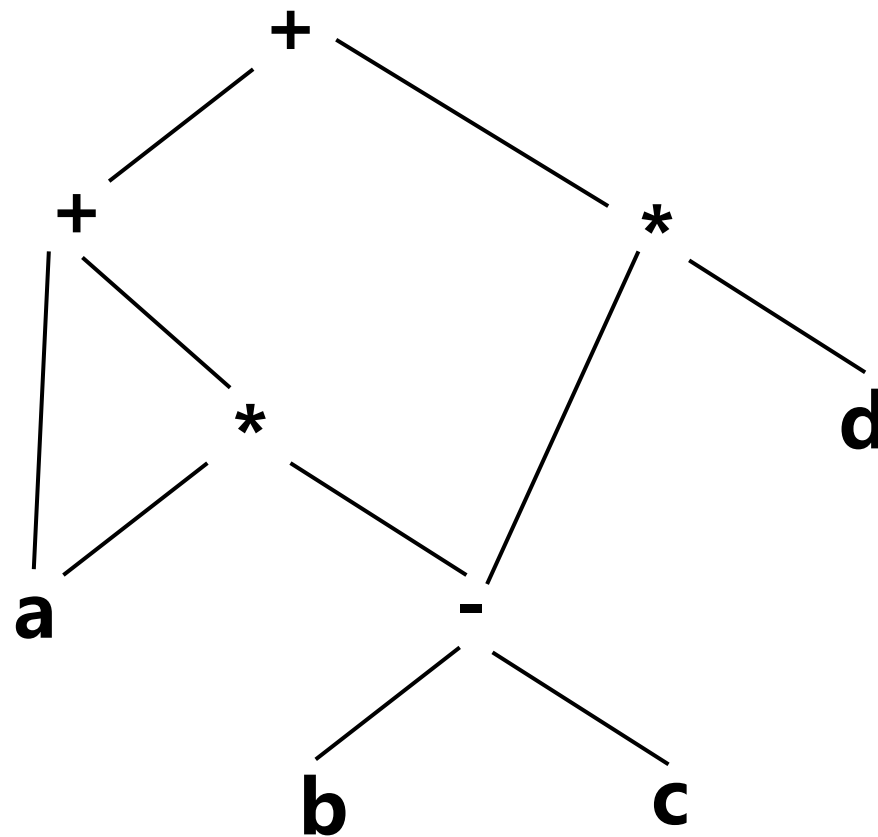
$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

$F \rightarrow num$



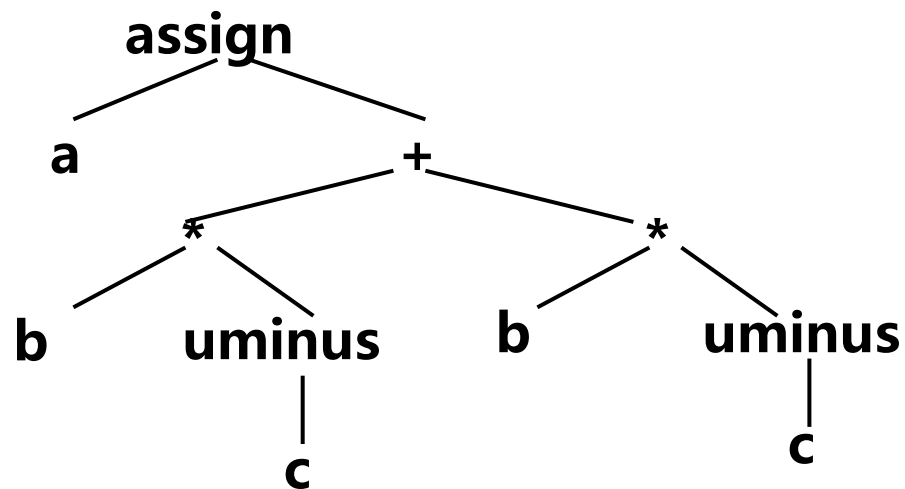
三地址代码

- 三地址代码

$x := y \text{ op } z$

- 三地址代码可以看成是抽象语法树或DAG的一种线性表示

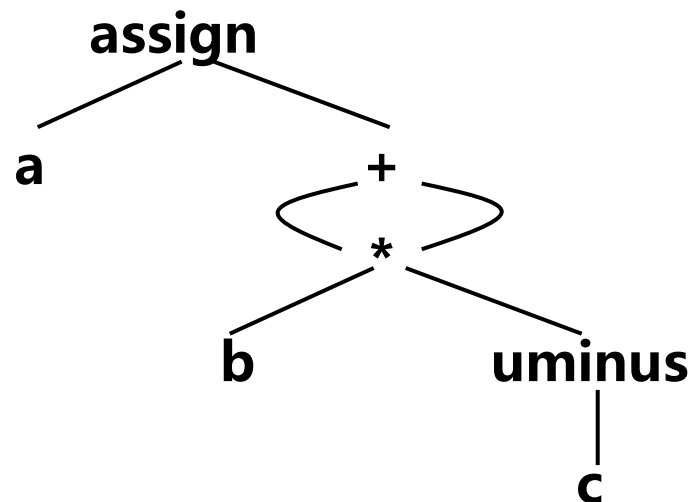
$a := b * (-c) + b * (-c)$ 的图表示法



抽象语法树

抽象语法树对应的代码:

```
T1 := -c  
T2 := b * T1  
T3 := -c  
T4 := b * T3  
T5 := T2 + T4  
a := T5
```



DAG

DAG对应的代码:

```
T1 := -c  
T2 := b * T1  
T5 := T2 + T2  
a := T5
```

三地址语句的种类

- $x := y \text{ op } z$
- $x := \text{op } y$
- $x := y$
- `goto L`
- `if x relop y goto L` 或 `if a goto L`
- `param x` 和 `call p, n`, 以及返回语句 `return y`
- $x := y[i]$ 及 $x[i] := y$ 的索引赋值
- $x := \&y$, $x := *y$ 和 $*x := y$ 的地址和指针赋值

语句的三地址代码

- 生成三地址代码时，临时变量的名字对应抽象语法树的内部结点
 - 产生式 $E \rightarrow E1 + E2$
 - E的值放到一个新的临时变量T中
 - 赋值语句 $id := E$ 的三地址代码包括：
 - (1) 对表达式E求值并置于变量T中
 - (2) 进行赋值 $id.place := T$

赋值语句的文法

产生式

$S \rightarrow id := E$

$E \rightarrow E_1 + E_2$

$E \rightarrow E_1 * E_2$

$E \rightarrow -E_1$

$E \rightarrow (E_1)$

$E \rightarrow id$

- 非终结符号S有综合属性S.code，它代表赋值语句S的三地址代码。
- 非终结符号E有如下两个属性：
 - E.place表示存放E值的名字。
 - E.code表示对E求值的三地址语句序列。
 - 函数newtemp的功能是，每次调用它时，将返回一个不同临时变量名字,如 $T_1, T_2, \dots$ 。
 - gen表示生成三地址语句
- 三地址语句序列往往是被存放到一个输出文件中

属性文法定义

产生式

$S \rightarrow \text{id} := E$

$E \rightarrow E_1 + E_2$

$E \rightarrow E_1 * E_2$

$E \rightarrow -E_1$

$E \rightarrow (E_1)$

$E \rightarrow \text{id}$

语义规则

$S.\text{code} := E.\text{code} \parallel \text{gen}(\text{id.place} \text{ ' := ' } E.\text{place})$

$E.\text{place} := \text{newtemp};$

$E.\text{code} := E_1.\text{code} \parallel E_2.\text{code} \parallel$
 $\text{gen}(E.\text{place} \text{ ' := ' } E_1.\text{place} \text{ ' + ' } E_2.\text{place})$

$E.\text{place} := \text{newtemp};$

$E.\text{code} := E_1.\text{code} \parallel E_2.\text{code} \parallel$
 $\text{gen}(E.\text{place} \text{ ' := ' } E_1.\text{place} \text{ ' * ' } E_2.\text{place})$

$E.\text{place} := \text{newtemp};$

$E.\text{code} := E_1.\text{code} \parallel$
 $\text{gen}(E.\text{place} \text{ ' := ' 'uminus' } E_1.\text{place})$

$E.\text{place} := E_1.\text{place};$

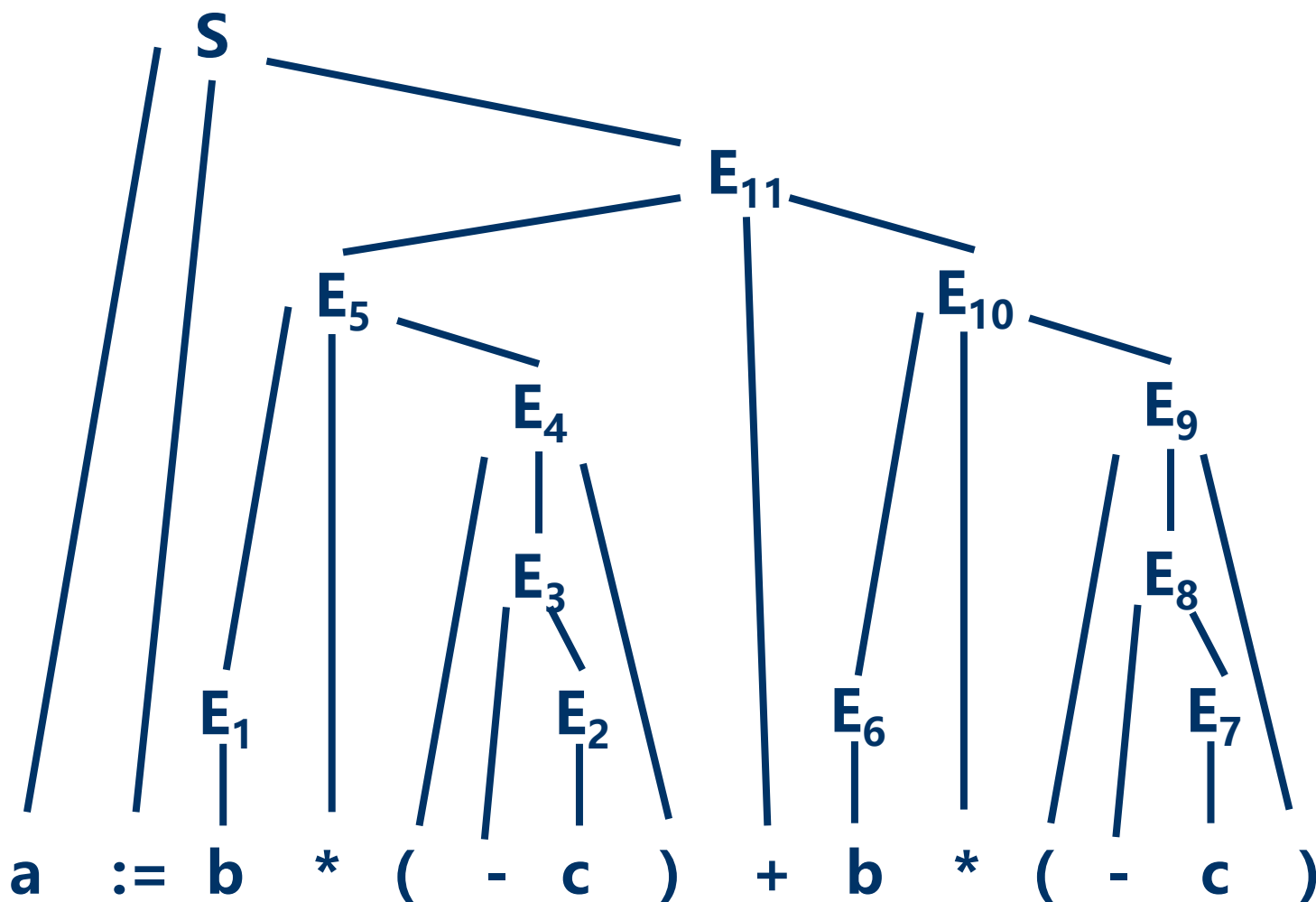
$E.\text{code} := E_1.\text{code}$

$E.\text{place} := \text{id.place};$

$E.\text{code} = \text{ ' ' }$

$a := b * (-c) + b * (-c)$ 三地址语句翻译

■ 严格翻译：两遍扫描



$T_1 := -c$

$T_2 := b * T_1$

$T_3 := -c$

$T_4 := b * T_3$

$T_5 := T_2 + T_4$

$a := T_5$

■ 四元式

$$a := b * (-c) + b * (-c)$$

- 一个带有四个域的记录结构，这四个域分别称为op, arg1, arg2及result

op	arg1	arg2	result
(0) uminus	c		T ₁
(1) *	b	T ₁	T ₂
(2) uminus	c		T ₃
(3) *	b	T ₃	T ₄
(4) +	T ₂	T ₄	T ₅
(5) :=	T ₅		a

- 四元式之间的联系通过临时变量实现。
- 单目运算只用arg1域，转移语句将目标标号放入result域。
- arg1,arg2,result通常为指针，指向有关名字的符号表入口，且临时变量填入符号表。

$$a := b * (-c) + b * (-c)$$

■ 三元式

- 通过计算临时变量值的语句的位置来引用这个临时变量
- 三个域: op、arg1和arg2

	op	arg1	arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

■ 间接三元式

- 为了便于代码优化，用三元式表+间接码表 表示中间代码
 - 间接码表:一张指示器表，按运算的先后次序列出有关三元式在三元式表中的位置。
- 优点: 方便优化，节省空间

示例

语句

$X := (A + B) * C;$

$Y := D \uparrow (A + B)$

的间接三元式表示如下表所示。

间接代码

(1)

(2)

(3)

(1)

(4)

(5)

三元式表

	OP	ARG1	ARG2
(1)	+	A	B
(2)	*	(1)	C
(3)	:=	X	(2)
(4)	↑	D	(1)
(5)	:=	Y	(4)

四元式、三元式和间接三元式比较

- 三元式中使用了指向三元式的指针，优化时修改较难。
- 间接三元式优化只需要更改间接码表，并节省三元式表存储空间。
- 修改四元式表也较容易，只是临时变量要填入符号表，占据一定存储空间。

内容线索

√. 中间语言

2. 说明语句的翻译

3. 赋值语句的翻译

4. 布尔表达式的翻译

5. 两遍扫描条件控制及其语句的翻译

6. 一遍扫描条件控制及其语句的翻译

7. 过程调用的处理

- 说明部分中把**定义性**出现的**标识符**与**类型等属性**相关联，从而确定它们在计算机**内部的表示法**、**取值范围**及可对其**进行的运算**。
- 为了产生有效地可执行目标代码，对于说明部分的翻译，不仅仅把与标识符相关联的类型等属性填入符号表中，还必须考虑到标识符所标记的对象的存储分配问题。

常量定义的翻译

- C++语言中有常量定义，以关键字const为标志，如

`const float pi=3.1416, one=1.0`

- 对每个常量定义的处理应包括下列工作：

(1) 把等号右边的常量值登录入常量表中（若之前尚未登录）并回送常量表序号；

(2) 然后为等号左边的标识符建立符号表新条目，在该条目中填入常量标志、相应类型及常量表序号。

- 假定常量定义中不指明类型，类型直接由常量值本身确定，且等号右边仅整数或常量标识符。

- 常量定义及相应翻译方案

CONSTEDF → const CDT

CDT → CDT; CD

CDT → CD

CD → id=num

```
{ num.ord:=lookCT(num.lexval)
  id.ord:=num.ord; id.type:=integer;
  id.kind:=CONSTANT
  add(id.entry, id.kind, id.type, id.ord)}
```

CD → id=id₁

```
{id.kind:=CONSTANT; id.type:=id1.type;
  id.ord:=id1.ord
  add(id.entry, id.kind, id.type, id.ord)}
```

lookCT(c)将在常量表中查找常量c，若查不到，则将该常量值登录入常量表。最终回送常量c值在常量表中的序号

过程add是对过程addtype的扩充，它把种类、类型与序号三者填入符号表相应条目中

变量说明的翻译

- 变量说明部分由一组变量说明语句组成，这里假定每个变量说明语句中仅包含一个标识符。
- 变量说明部分的语法定义

$P \rightarrow D;$

$D \rightarrow D; D$

$D \rightarrow \text{id}: T$

$T \rightarrow \text{integer}$

$T \rightarrow \text{real}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$

$T \rightarrow \uparrow T_1$

➤ 变量说明部分的翻译

$P \rightarrow D; \quad \{\text{offset}:=0\}$

$D \rightarrow D; D$

$D \rightarrow \text{id}: T$

$\{\text{enter}(\text{id.name}, T.\text{type}, \text{offset});$
 $\text{offset}:=\text{offset}+T.\text{width}\}$

$T \rightarrow \text{integer}$

$\{T.\text{type}:=\text{integer}; T.\text{width}:=4\}$

$T \rightarrow \text{real}$

$\{T.\text{type}:=\text{real}; T.\text{width}:=8\}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$

$\{T.\text{type}:=\text{array}(\text{num.val}, T_1.\text{type});$
 $T.\text{width}:=\text{num.val}*T_1.\text{width}\}$

$T \rightarrow \uparrow T_1$

$\{T.\text{type}:=\text{pointer}(T_1.\text{type}); T.\text{width}:=4\}$

■ $P \rightarrow D; \quad \{\text{offset}:=0\}$

为了使得**offset**赋初值更明显

变更为 $P \rightarrow \{\text{offset}:=0\} D;$

也可采用增加非终结符号及产生式来实现

$$P \rightarrow M D$$
$$M \rightarrow \varepsilon \{\text{offset}:=0\}$$

■ 过程（函数）定义的语法

$P \rightarrow D;$

$D \rightarrow D; D$

$D \rightarrow \text{id}: T$

$D \rightarrow \text{proc id}; D; S$

$T \rightarrow \text{integer}$

$T \rightarrow \text{real}$

$T \rightarrow \text{array[num] of } T_1$

$T \rightarrow \uparrow T_1$

■ 过程及函数定义的翻译必然涉及标识符作用域问题

■ 基本思想：每个过程有一张独立的符号表

过程示例

```
(1) Program sort(input, output)
(2)  var a: array[0..10] of integer;
(3)      x: integer;
(4)      procedure readarray;
(5)          var i: integer;
(6)          begin ...a ... end {readarray}
(7)      procedure exchange(i, j:integer)
(8)          begin
(9)              x:=a[i]; a[i]:=a[j]; a[j]:=x
(10)         end {exchange}
(11)     procedure quicksort (m, n:integer);
(12)         var k, v: integer;
(13)         function partition (y,z: integer): integer;
(14)             var i,j: integer;
(15)             begin ...a ...
(16)                 ...v ...
(17)                 ...exchange(i, j); ...
(18)             end {partition}
(19)         begin ... end {quicksort};
(20) begin ... end {sort}.
```

■ 语义规则中的操作

- **mktable(previous)** : 创建一张新符号表, 并返回指向新表的一个指针。
 - 参数previous指向一张先前创建的符号表, 其值放在新符号表表头
- **enter(table, name, type, offset)**: 在指针table指示的符号表中为名字name建立一个新项, 并把类型type、相对地址offset填入到该项中。
- **addwidth(table, with)**: 指针table指示的符号表表头中记录下该表中所有名字占用的总宽度
- **enterproc(table, name, newtable)**: 在指针table指示的符号表中为名字name的过程建立一个新项。
 - 参数newtable指向过程name的符号表

■ 翻译

$P \rightarrow M D;$

$M \rightarrow \varepsilon$

$D \rightarrow D_1; D_2$

$D \rightarrow \text{proc id}; N D_1; S$

$D \rightarrow \text{id}: T$

$N \rightarrow \varepsilon$

```
{addwidth(top(tblptr), top(offset));  
  pop(tblptr); pop(offset)}
```

```
{t:=mktable(nil);  
  push(t, tblptr); push(0, offset)}
```

```
{t:=top(tblptr);  
  addwidth(t, top(offset));  
  pop(tblptr); pop(offset)  
  enterproc(top(tblptr), id.name, t)}
```

```
{enter(top(tblptr), id.name, T.type,  
  top(offset));  
  top(offset):=top(offset)+T.width}
```

```
{t:=mktable(top(tblptr));  
  push(t, tblptr); push(0, offset)}
```

- 栈tblptr 保存各外层过程的符号表指针
- 栈offset 存放各嵌套过程的当前相对地址

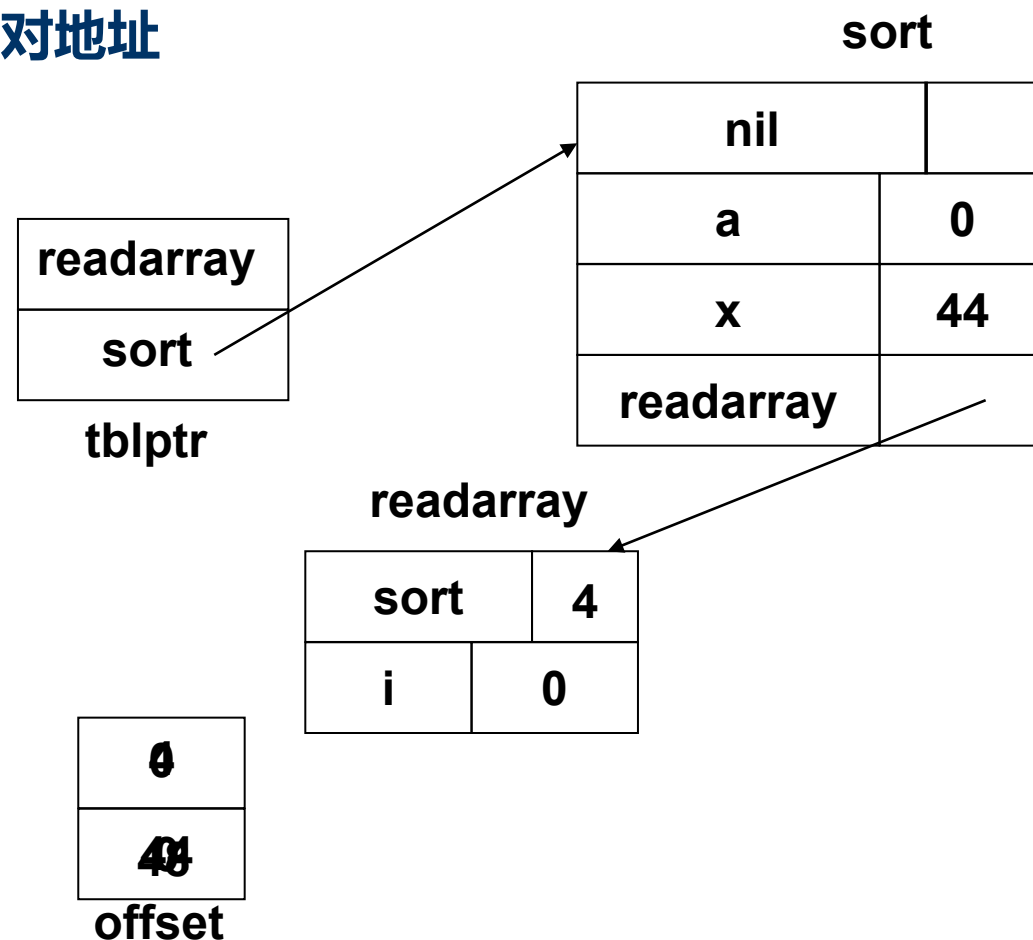
➤ 栈顶元素为当前被处理过程的下一个名字的相对地址

```

(1) Program sort(input, output)
(2)  var a: array[0..10] of integer;
(3)      x: integer;
(4)      procedure readarray;
(5)          var i: integer;
(6)          begin ...a ... end {readarray}

```

.....



内容线索

√. 中间语言

√. 说明语句的翻译

3. 赋值语句的翻译

4. 布尔表达式的翻译

5. 两遍扫描条件控制及其语句的翻译

6. 一遍扫描条件控制及其语句的翻译

7. 过程调用的处理

赋值语句两遍扫描翻译的属性文法

产生式

$S \rightarrow \text{id} := E$

$E \rightarrow E_1 + E_2$

$E \rightarrow E_1 * E_2$

$E \rightarrow -E_1$

$E \rightarrow (E_1)$

$E \rightarrow \text{id}$

语义规则

$S.\text{code} := E.\text{code} \parallel \text{gen}(\text{id.place} \text{ ' := ' } E.\text{place})$

$E.\text{place} := \text{newtemp};$

$E.\text{code} := E_1.\text{code} \parallel E_2.\text{code} \parallel$
 $\text{gen}(E.\text{place} \text{ ' := ' } E_1.\text{place} \text{ ' + ' } E_2.\text{place})$

$E.\text{place} := \text{newtemp};$

$E.\text{code} := E_1.\text{code} \parallel E_2.\text{code} \parallel$
 $\text{gen}(E.\text{place} \text{ ' := ' } E_1.\text{place} \text{ ' * ' } E_2.\text{place})$

$E.\text{place} := \text{newtemp};$

$E.\text{code} := E_1.\text{code} \parallel$
 $\text{gen}(E.\text{place} \text{ ' := ' 'uminus' } E_1.\text{place})$

$E.\text{place} := E_1.\text{place};$

$E.\text{code} := E_1.\text{code}$

$E.\text{place} := \text{id.place};$

$E.\text{code} = \text{ ' ' }$

■ 简单算术表达式及赋值语句

➤ 简单算术表达式及赋值语句翻译为三地址代码的翻译模式

- 属性`id.name` 表示`id`所代表的名字本身
- 过程`lookup(id.name)`检查是否在符号表中存在相应此名字的入口。
如果有，则返回一个指向该表项的指针，否则，返回`nil`表示没有找到
- 过程`emit`将生成的三地址语句发送到输出文件中

赋值语句一遍扫描翻译模式

$S \rightarrow id := E$	$S.code := E.code \parallel gen(id.place \text{ '}' := \text{' } E.place)$
$E \rightarrow E_1 + E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place \text{ '}' := \text{' } E_1.place \text{ '}' + \text{' } E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place \text{ '}' := \text{' } E_1.place \text{ '}' * \text{' } E_2.place)$

$S \rightarrow id := E$	{ $p := lookup(id.name);$ if $p \neq nil$ then emit($p \text{ '}' := \text{' } E.place$) else error }
-------------------------	--

$E \rightarrow E_1 + E_2$	{ $E.place := newtemp;$ emit($E.place \text{ '}' := \text{' } E_1.place \text{ '}' + \text{' } E_2.place$)}
---------------------------	--

$E \rightarrow E_1 * E_2$	{ $E.place := newtemp;$ emit($E.place \text{ '}' := \text{' } E_1.place \text{ '}' * \text{' } E_2.place$)}
---------------------------	--

赋值语句一遍扫描翻译模式

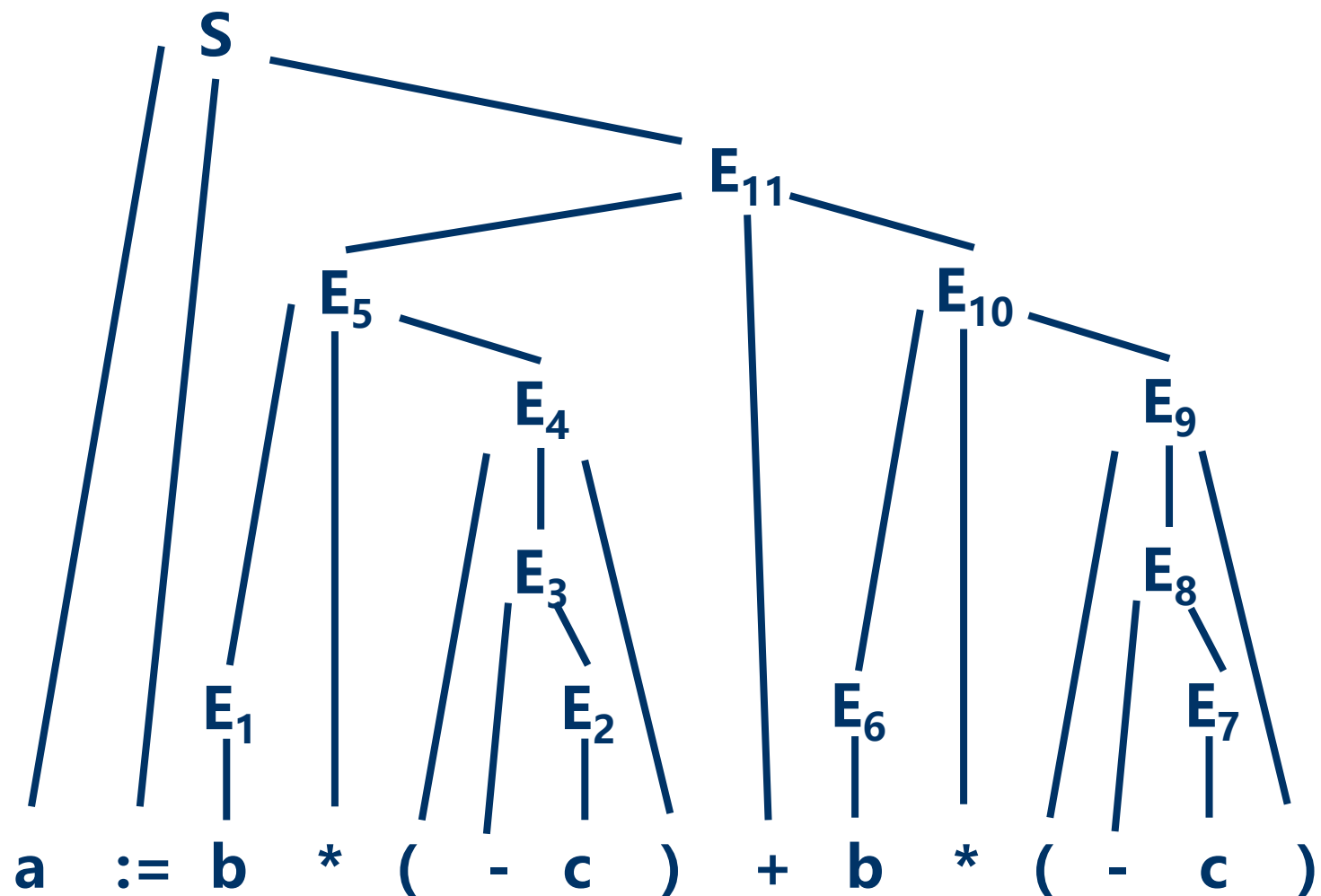
$E \rightarrow -E_1$	$E.place := newtemp;$ $E.code := E_1.code \parallel \text{gen}(E.place \text{ ':=' } 'uminus' E_1.place)$
$E \rightarrow (E_1)$	$E.place := E_1.place;$ $E.code := E_1.code$
$E \rightarrow id$	$E.place := id.place;$ $E.code = ''$

$E \rightarrow -E_1$	{ $E.place := newtemp;$ $\text{emit}(E.place \text{ ':=' } 'uminus' E_1.place)$ }
----------------------	--

$E \rightarrow (E_1)$	{ $E.place := E_1.place$ }
-----------------------	----------------------------

$E \rightarrow id$	{ $p := \text{lookup}(id.name);$ if $p \neq \text{nil}$ then $E.place := p$ else error }
--------------------	---

$a := b * (-c) + b * (-c)$ —遍翻译



$T_1 := -c$

$T_2 := b * T_1$

$T_3 := -c$

$T_4 := b * T_3$

$T_5 := T_2 + T_4$

$a := T_5$

类型转换

- 用E.type表示非终结符E的类型属性
- 对应产生式 $E \rightarrow E_1 \text{ op } E_2$ 的语义动作中关于E.type的语义规则可定义为：
 { if $E_1.\text{type} = \text{integer}$ and $E_2.\text{type} = \text{integer}$
 E.type:=integer
 else E.type:=real }
- 进行类型转换的三地址代码
 x:= inttoreal y
- 算符区分为整型算符int op和实型算符real op,

例. $x := y + i * j$

其中x、y为实型；i、j为整型。这个赋值句产生的三地址代码为：

```
T1 := i int* j
T2 := inttoreal T1
T3 := y real+ T2
x := T3
```

关于产生式 $E \rightarrow E_1 + E_2$ 的语义动作

```
{ E.place:=newtemp;  
if E1.type=integer and E2.type=integer then begin  
    emit (E.place ':=' E1.place 'int+' E2.place);  
    E.type:=integer  
end  
else if E1.type=real and E2.type=real then begin  
    emit (E.place ':=' E1.place 'real+' E2.place);  
    E.type:=real  
end  
else if E1.type=integer and E2.type=real then begin  
    u:=newtemp;  
    emit (u ':=' 'inttoreal' E1.place);  
    emit (E.place ':=' u 'real+' E2.place);  
    E.type:=real  
end  
else if E1.type=real and E2.type=integer then begin  
    u:=newtemp;  
    emit (u ':=' 'inttoreal' E2.place);  
    emit (E.place ':=' E1.place 'real+' u);  
    E.type:=real  
end  
else E.type:=type_error}
```

内容线索

√. 中间语言

√. 说明语句的翻译

√. 赋值语句的翻译

4. 布尔表达式的翻译

5. 两遍扫描条件控制及其语句的翻译

6. 一遍扫描条件控制及其语句的翻译

7. 过程调用的处理

布尔表达式的翻译

- **布尔表达式**：用**布尔运算符**把**布尔量**、**关系表达式**联结起来的式子。

- 布尔运算符：and, or, not ;
- 关系运算符 <, ≤, =, ≠, >, ≥

- **布尔表达式的两个基本作用**：

- 用于逻辑演算，计算逻辑值；
- 用于控制语句的条件式。

- **产生布尔表达式的文法**：

- $E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \neg E \mid (E) \mid \text{id rop id} \mid \text{id}$

- **运算符优先级**：布尔运算由高到低：not and or，同级左结合

关系运算符同级，且高于布尔运算符

- 计算布尔表达式如同计算算术表达式一样，一步步算

$1 \text{ or } (\text{not } 0 \text{ and } 0) \text{ or } 0$

$= 1 \text{ or } (1 \text{ and } 0) \text{ or } 0$

$= 1 \text{ or } 0 \text{ or } 0$

$= 1 \text{ or } 0$

$= 1$

- **a or b and not c 翻译成**

$T_1 := \text{not } c$

$T_2 := b \text{ and } T_1$

$T_3 := a \text{ or } T_2$

- **a < b 的关系表达式可等价地写成**

if a < b then 1 else 0 , 翻译成

100: if a < b goto 103

101: T:=0

102: goto 104

103: T:=1

104:

数值表示法的翻译模式

- 过程emit将三地址代码送到输出文件中
- nextstat: 给出输出序列中下一条三地址语句的地址索引
- 每产生一条三地址语句后, 过程emit便把nextstat加1

数值表示法的翻译模式

$E \rightarrow E_1 \text{ or } E_2$ $\{E.place := \text{newtemp};$
 $\text{emit}(E.place \text{ ':='} E_1.place \text{ 'or'} E_2.place)\}$

$E \rightarrow E_1 \text{ and } E_2$ $\{E.place := \text{newtemp};$
 $\text{emit}(E.place \text{ ':='} E_1.place \text{ 'and'} E_2.place)\}$

$E \rightarrow \text{not } E_1$ $\{E.place := \text{newtemp};$
 $\text{emit}(E.place \text{ ':='} \text{ 'not'} E_1.place)\}$

$E \rightarrow (E_1)$ $\{E.place := E_1.place\}$

数值表示法的翻译模式

a < b 翻译成

```
100:  if a < b goto 103
101:  T:=0
102:  goto 104
103:  T:=1
104:
```

$E \rightarrow id_1 \text{ relop } id_2$

{ E.place:=newtemp;

emit('if' id₁.place relop.op
id₂.place 'goto' nextstat+3);

emit(E.place ':=' '0');

emit('goto' nextstat+2);

emit(E.place ':=' '1') }

$E \rightarrow id$

{ E.place:=id.place }

布尔表达式 $a < b$ or $c < d$ and $e < f$ 的翻译结果

100: if $a < b$ goto 103

101: $T_1 := 0$

102: goto 104

103: $T_1 := 1$

104: if $c < d$ goto 107

105: $T_2 := 0$

106: goto 108

107: $T_2 := 1$

108: if $e < f$ goto 111

109: $T_3 := 0$

110: goto 112

111: $T_3 := 1$

112: $T_4 := T_2$ and T_3

113: $T_5 := T_1$ or T_4

```
E → id1 relop id2
{ E.place := newtemp;
  emit( 'if' id1.place relop.op
        id2.place 'goto' nextstat+3);
  emit(E.place ':=' '0' );
  emit( 'goto' nextstat+2);
  emit(E.place ':=' '1' ) }
```

```
E → id
{ E.place := id.place }
```

```
E → E1 or E2
{ E.place := newtemp;
  emit(E.place ':=' E1.place 'or' E2.place) }
```

```
E → E1 and E2
{ E.place := newtemp;
  emit(E.place ':=' E1.place 'and' E2.place) }
```

内容线索

√. 中间语言

√. 说明语句的翻译

√. 赋值语句的翻译

√. 布尔表达式的翻译

5. 两遍扫描条件控制及其语句的翻译

6. 一遍扫描条件控制及其语句的翻译

7. 过程调用的处理

作为条件控制的布尔式翻译

■ 采用优化措施

- 把A or B解释成 `if A then true else B`
- 把A and B解释成 `if A then B else false`
- 把 $\neg A$ 解释成 `if A then false else true`

作为条件控制的布尔式翻译

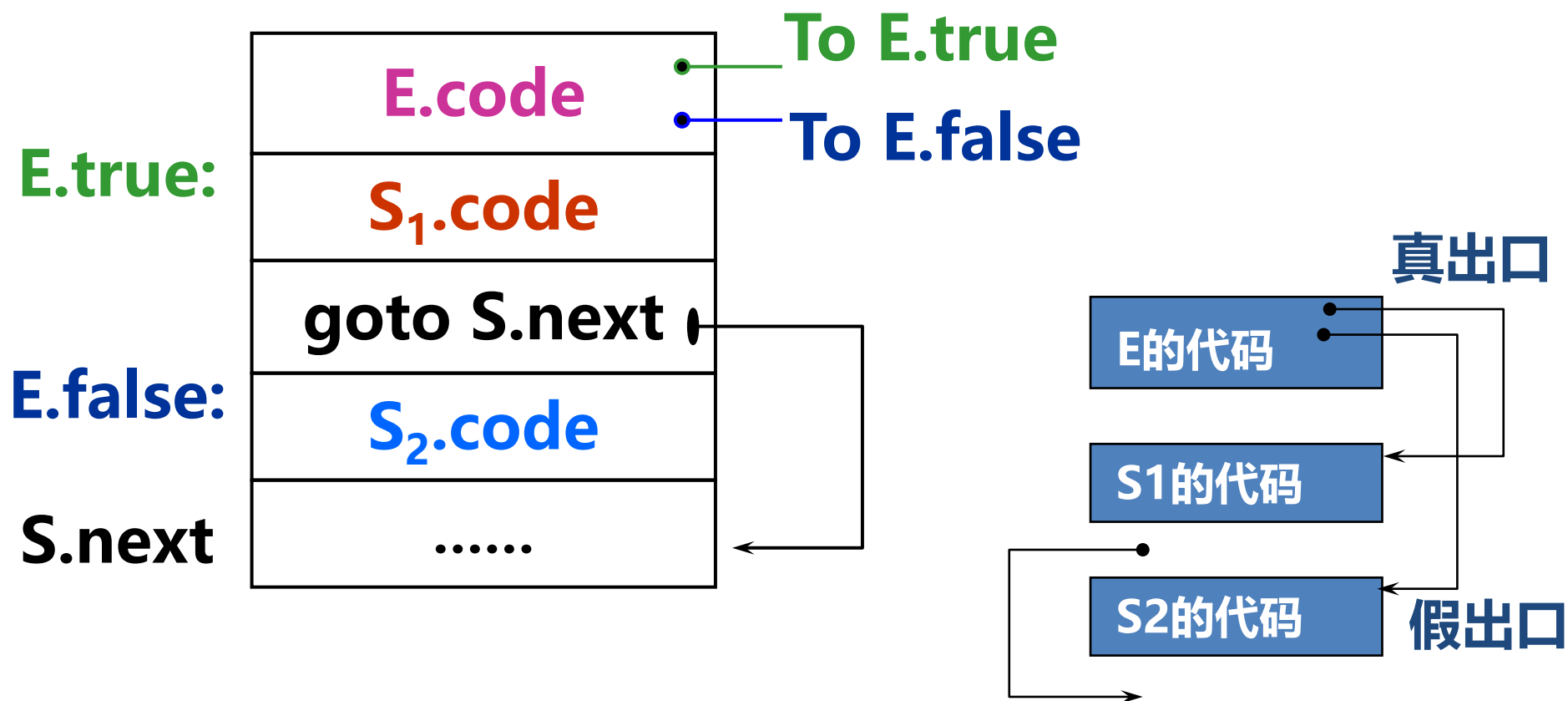
■ 两遍扫描

- 为给定的输入串构造一棵语法树；
- 对语法树进行深度优先遍历，进行语义规则中规定的翻译。

■ 一遍扫描

作为条件控制的布尔式翻译

- 条件语句 if E then S_1 else S_2
赋予 E 两种出口：一真一假



布尔式翻译的基本思想

- 对于一个布尔表达式E，引用两个标号：
 - E.true : E为真时控制流转向的标号
 - E.false : E为假时控制流转向的标号
- 假定E形如 $a < b$ ，则将生成如下的E的代码：

if $a < b$ goto E.true

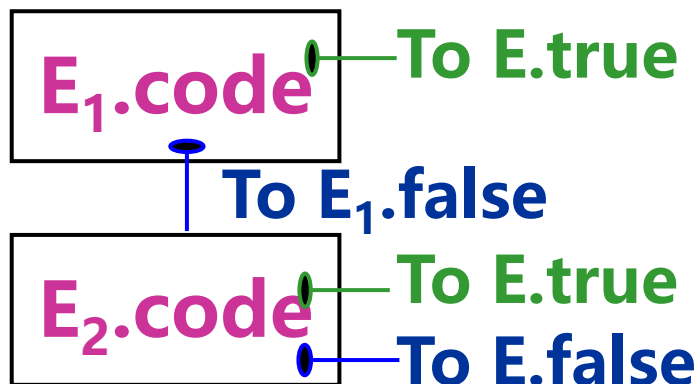
goto E.false

产生布尔表达式三地址代码的语义规则

产生式

$E \rightarrow E_1 \text{ or } E_2$

E.true是E为‘真’
时控制流转向的标号



语义规则

$E_1.true := E.true;$

$E_1.false := newlabel;$

$E_2.true := E.true;$

$E_2.false := E.false;$

$E.code := E_1.code \parallel$

$gen(E_1.false \text{ ':' }) \parallel E_2.code$

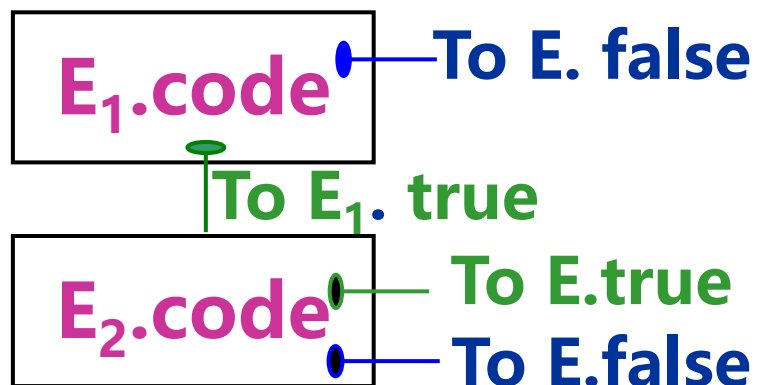
每次调用函数
newlabel后都返回
一个新的符号标号

E.false是E为‘假’
时控制流转向的标号

产生布尔表达式三地址代码的语义规则

产生式

$E \rightarrow E_1 \text{ and } E_2$



语义规则

$E_1.true := \text{newlabel};$

$E_1.false := E.false;$

$E_2.true := E.true;$

$E_2.false := E.false;$

$E.code := E_1.code \parallel$

$\text{gen}(E_1.true \text{ ':' }) \parallel E_2.code$

产生布尔表达式三地址代码的语义规则

产生式

$E \rightarrow \text{not } E_1$

$E \rightarrow (E_1)$

语义规则

$E_1.\text{true} := E.\text{false};$
 $E_1.\text{false} := E.\text{true};$
 $E.\text{code} := E_1.\text{code}$

$E_1.\text{true} := E.\text{true};$
 $E_1.\text{false} := E.\text{false};$
 $E.\text{code} := E_1.\text{code}$

产生布尔表达式三地址代码的语义规则

产生式

$E \rightarrow id_1 \text{ relop } id_2$

$E \rightarrow \text{true}$

$E \rightarrow \text{false}$

语义规则

$E.\text{code} := \text{gen}(\text{'if' } id_1.\text{place}$
 $\text{relop.op } id_2.\text{place 'goto' } E.\text{true})$
 $\parallel \text{gen}(\text{'goto' } E.\text{false})$

$E.\text{code} := \text{gen}(\text{'goto' } E.\text{true})$

$E.\text{code} := \text{gen}(\text{'goto' } E.\text{false})$

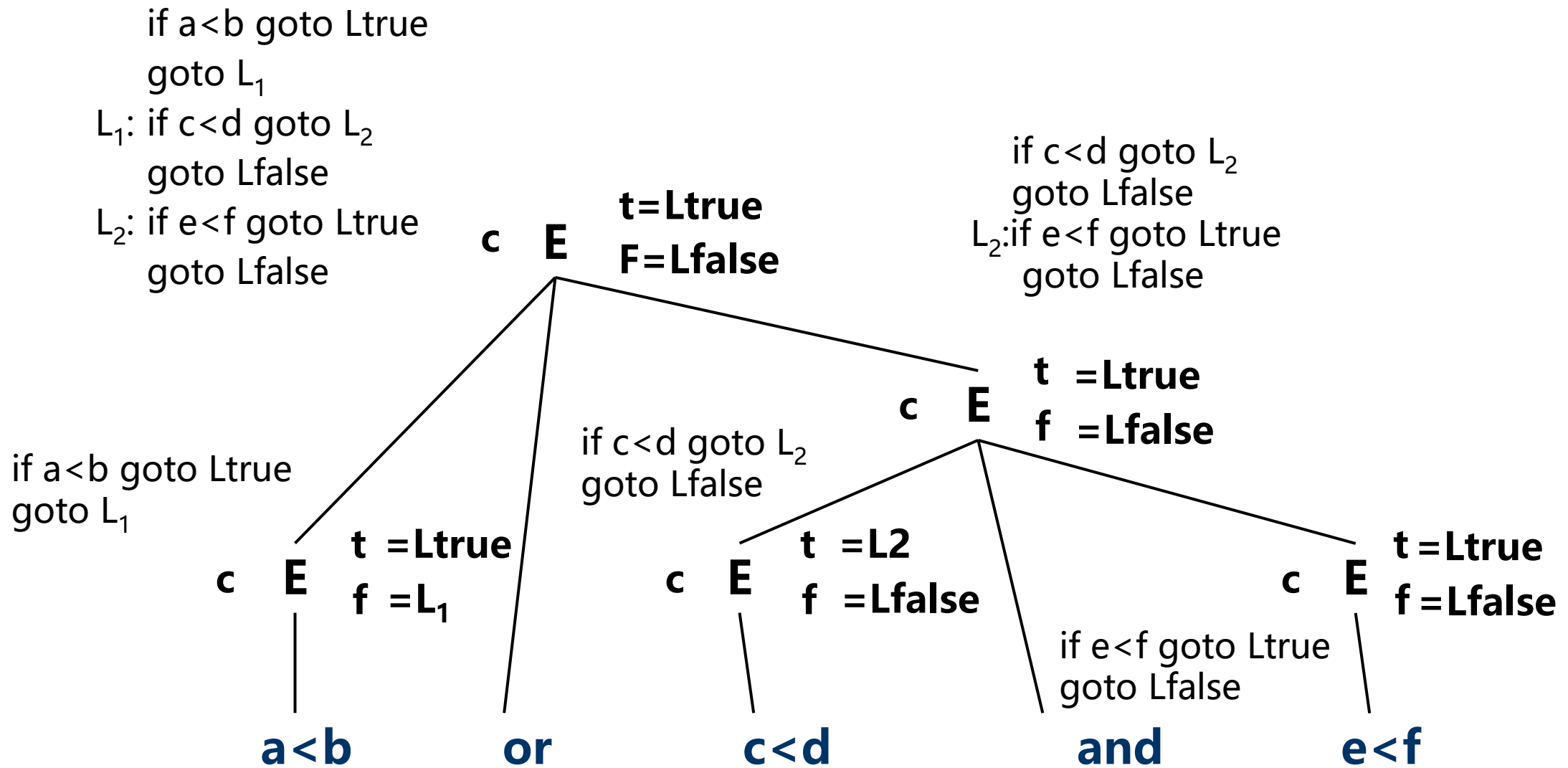
■ 考虑如下表达式:

$a < b \text{ or } c < d \text{ and } e < f$

假定整个表达式的真假出口已置为Ltrue和Lfalse

按照两遍扫描, 将生成如下的代码:

```
if a < b goto Ltrue
goto L1
L1: if c < d goto L2
      goto Lfalse
L2: if e < f goto Ltrue
      goto Lfalse
```



■ 考虑下列产生式所定义的语句

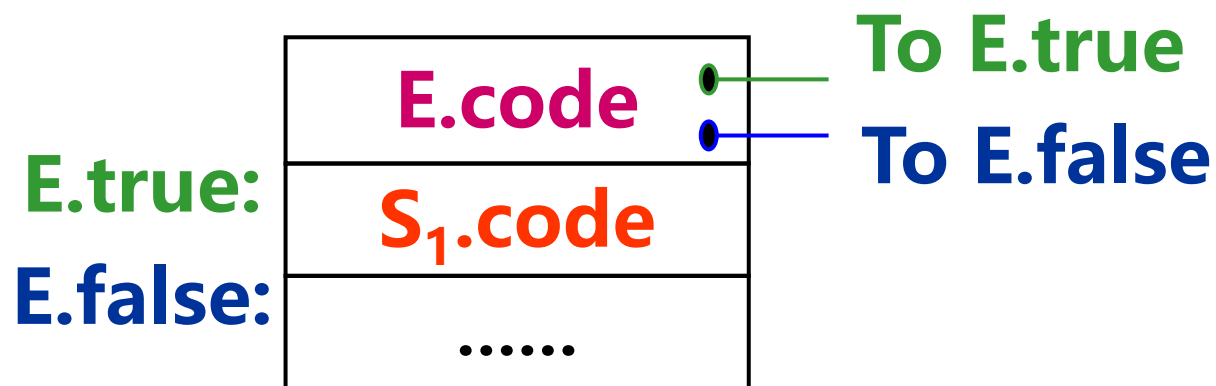
$S \rightarrow \text{if } E \text{ then } S_1$

| $\text{if } E \text{ then } S_1 \text{ else } S_2$

| $\text{while } E \text{ do } S_1$

其中E为布尔表达式。

if-then语句的属性文法



产生式

$S \rightarrow \text{if } E \text{ then } S_1$

S.next之值是一个标号，它指出继S的代码之后将被执行的第一条三地址指令

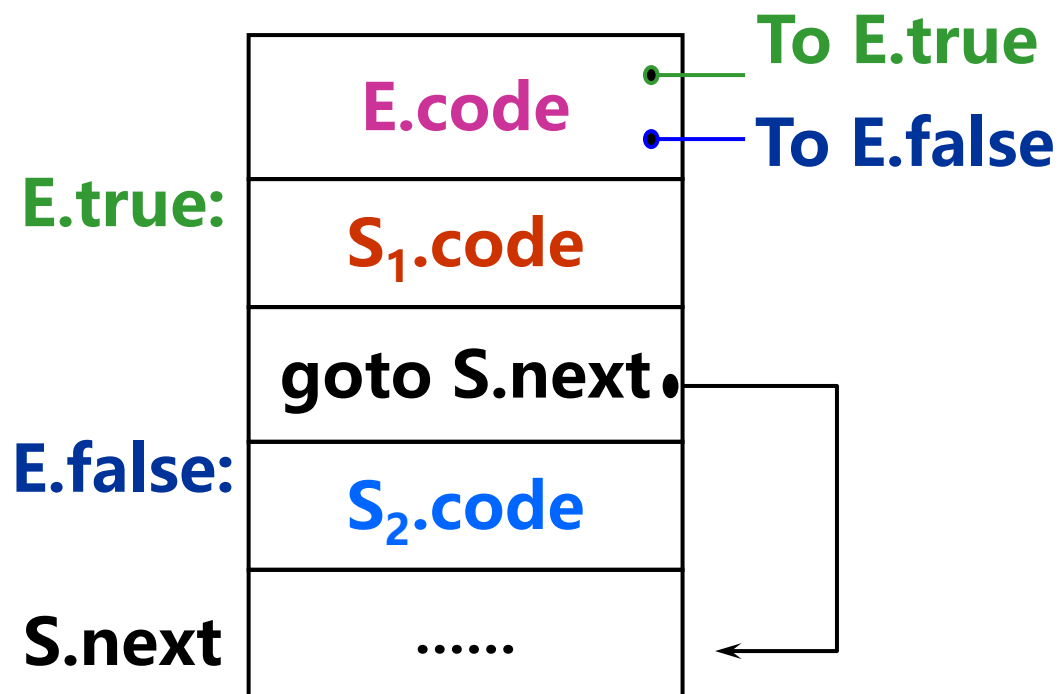
语义规则

```
E.true:=newlabel;  
E.false:=S.next;  
S1.next:=S.next  
S.code:=E.code ||  
gen(E.true ':') || S1.code
```

if-then-else语句的属性文法

产生式

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$



语义规则

```
E.true:=newlabel;  
E.false:=newlabel;  
S1.next:=S.next  
S2.next:=S.next;  
S.code:=E.code ||  
gen(E.true ':' ) || S1.code ||  
gen( 'goto' S.next) ||  
gen(E.false ':' ) || S2.code
```


示例

把语句: if $a > c$ or $b < d$ then S_1 else S_2
翻译成三地址代码, 其中S.next初始化为Lnext

if $a > c$ goto L1

goto L3

L3: if $b < d$ goto L1

goto L2

L1: (关于 S_1 的三地址代码序列)

goto Lnext

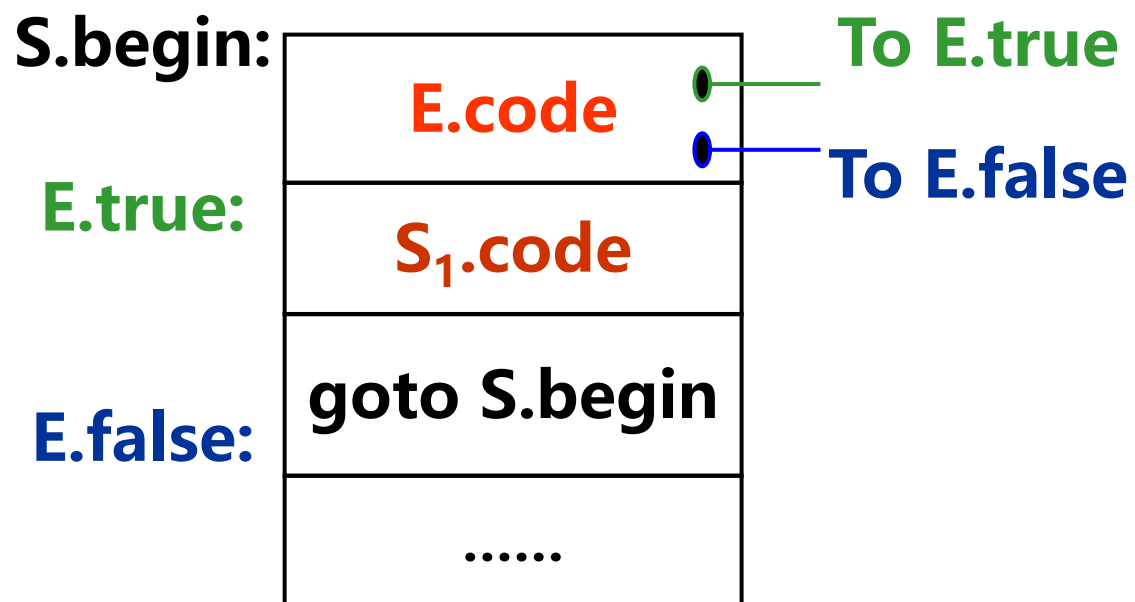
L2: (关于 S_2 的三地址代码序列)

Lnext:

while-do语句的属性文法

产生式

$S \rightarrow \text{while } E \text{ do } S_1$



语义规则

```
S.begin:=newlabel;  
E.true:=newlabel;  
E.false:=S.next;  
 $S_1$ .next:=S.begin;  
S.code:=gen(S.begin ':' ) ||  
E.code ||  
gen(E.true ':' ) ||  $S_1$ .code ||  
gen( 'goto' S.begin)
```

■ 考虑如下语句：

```
while a<b do
  if c<d then
    x:=y+z
  else
    x:=y-z
```

```
E→id1 relop id2
  E.code:=gen( 'if ' id1.place relop.op
               id2.place 'goto' E.true)
             ||gen( 'goto' E.false)
```

生成下列代码：

```
L1:      if a<b goto L2
          goto Lnext
L2:      if c<d goto L3
          goto L4
L3:      T1:=y+z
          x:=T1
          goto L1
L4:      T2:=y-z
          x:=T2
          goto L1
Lnext:
```

内容线索

√. 中间语言

√. 说明语句的翻译

√. 赋值语句的翻译

√. 布尔表达式的翻译

√. 两遍扫描条件控制及其语句的翻译

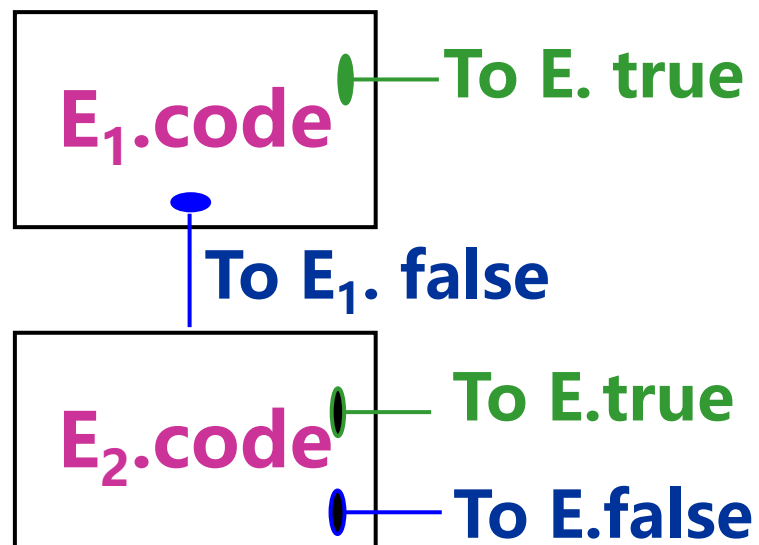
6. 一遍扫描条件控制及其语句的翻译

7. 过程调用的处理

两遍扫描回顾

产生式

$E \rightarrow E_1 \text{ and } E_2$



语义规则

$E_1.true := \text{newlabel};$

$E_1.false := E.false;$

$E_2.true := E.true;$

$E_2.false := E.false;$

$E.code := E_1.code \parallel$

$\text{gen}(E_1.true \text{ ':' }) \parallel E_2.code$

一遍扫描实现布尔表达式的翻译

- 采用四元式形式
- 把四元式存入一个数组中，数组下标就代表四元式的标号
- 约定

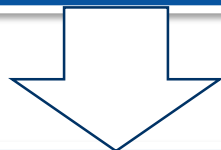
四元式(jnz, a, -, p) 表示 if a goto p

四元式(jrop, x, y, p) 表示 if x rop y goto p

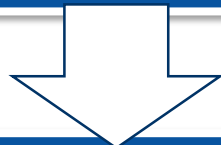
四元式(j, -, -, p) 表示 goto p

一遍扫描实现布尔表达式的翻译

没有语法树



问题：信息不全



四元式转移地址无法立即知道，
我们只好把这个未完成的四元式地址
作为E的语义值保存，待机“回填”。

- 为非终结符E赋予两个综合属性
E.truelist和E.falselist。它们分别记录布尔表达式E所对应的四元式中需回填“真”、“假”出口的四元式的标号所构成的链表
- 例如: 假定E的四元式中需要回填“真”出口的p, q, r三个四元式, 则E.truelist为下列链:



- 为了处理E.truelist和E.falselist，引入下列语义变量和过程：
 - 变量nextquad，它指向下一条将要产生但尚未形成的四元式的地址(标号)。nextquad的初值为1，每当执行一次emit之后，nextquad将自动增1。
 - 函数makelist(i)，它将创建一个仅含i的新链表，其中i是四元式数组的一个下标(标号)；函数返回指向这个链的指针。
 - 函数merge(p_1, p_2)，把以 p_1 和 p_2 为链首的两条链合并为一，作为函数值，回送合并后的链首。
 - 过程backpatch(p, t)，其功能是完成“回填”，把p所链接的每个四元式的第四区段都填为t。

布尔表达式的文法

- (1) $E \rightarrow E_1 \text{ or } M E_2$
- (2) $\quad \quad \quad | E_1 \text{ and } M E_2$
- (3) $\quad \quad \quad | \text{ not } E_1$
- (4) $\quad \quad \quad | (E_1)$
- (5) $\quad \quad \quad | \text{id}_1 \text{ relop id}_2$
- (6) $\quad \quad \quad | \text{id}$
- (7) $M \rightarrow \varepsilon$

布尔表达式的翻译模式

(5) $E \rightarrow id_1 \text{ relop } id_2$

```
{ E.truelist:=makelist(nextquad);  
  E.falselist:=makelist(nextquad+1);  
  emit( 'j' relop.op ',' id1.place ',' id2.place ',' '0' );  
  emit( 'j, -, -, 0' ) }
```

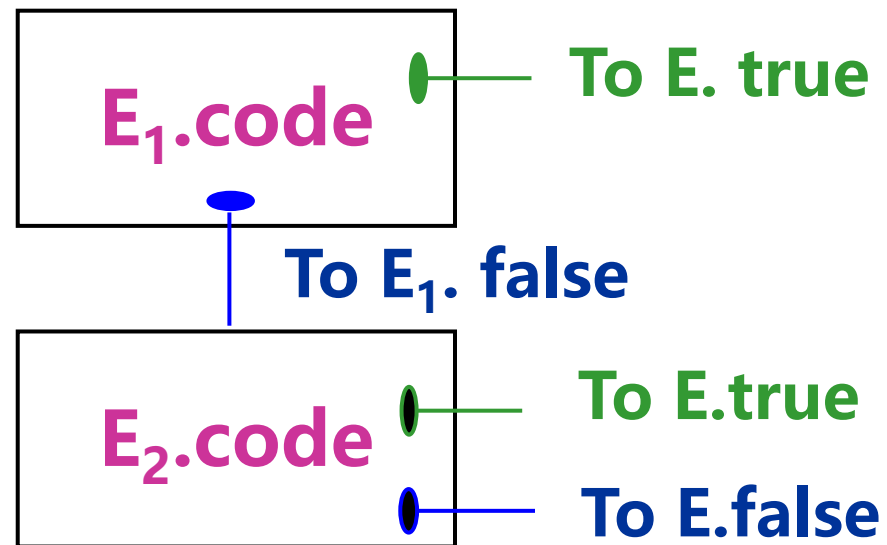
(6) $E \rightarrow id$

```
{ E.truelist:=makelist(nextquad);  
  E.falselist:=makelist(nextquad+1);  
  emit( 'jnz' ',' id.place ',' '-' ',' '0' );  
  emit( 'j, -, -, 0' ) }
```

(7) $M \rightarrow \varepsilon$

```
{ M.quad:=nextquad }
```

布尔表达式的翻译模式



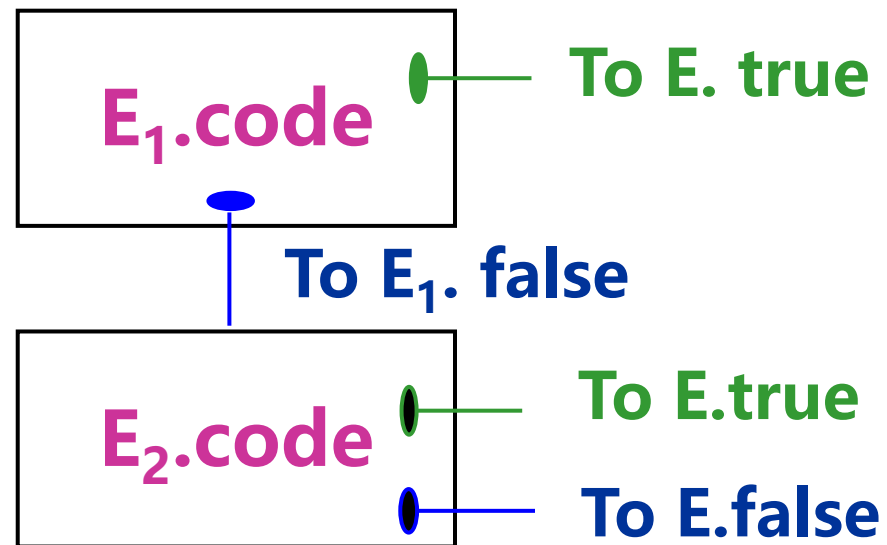
(1) $E \rightarrow E_1 \text{ or } E_2$

```
{ backpatch( $E_1$ .falselist, M.quad);
```

```
  E.truelist:=merge( $E_1$ .truelist,  $E_2$ .truelist);
```

```
  E.falselist:= $E_2$ .falselist }
```

布尔表达式的翻译模式



(2) $E \rightarrow E_1$ and $M E_2$

```
{ backpatch( $E_1$ .truelist, M.quad);
```

```
   $E$ .truelist:= $E_2$ .truelist;
```

```
   $E$ .falselist:=merge( $E_1$ .falselist, $E_2$ .falselist) }
```

布尔表达式的翻译模式

(3) $E \rightarrow \text{not } E_1$

**{ $E.\text{truelist} := E_1.\text{falselist};$
 $E.\text{falselist} := E_1.\text{truelist}$ }**

(4) $E \rightarrow (E_1)$

**{ $E.\text{truelist} := E_1.\text{truelist};$
 $E.\text{falselist} := E_1.\text{falselist}$ }**

$a < b$ or $c < d$ and $e < f$

作为整个布尔表达式的"真""假"
出口(转移目标)仍待回填

100 (j<, a, b, 0)

101 (j, -, -, 102)

102 (j<, c, d, 104)

103 (j, -, -, 0)

104 (j<, e, f, 100)

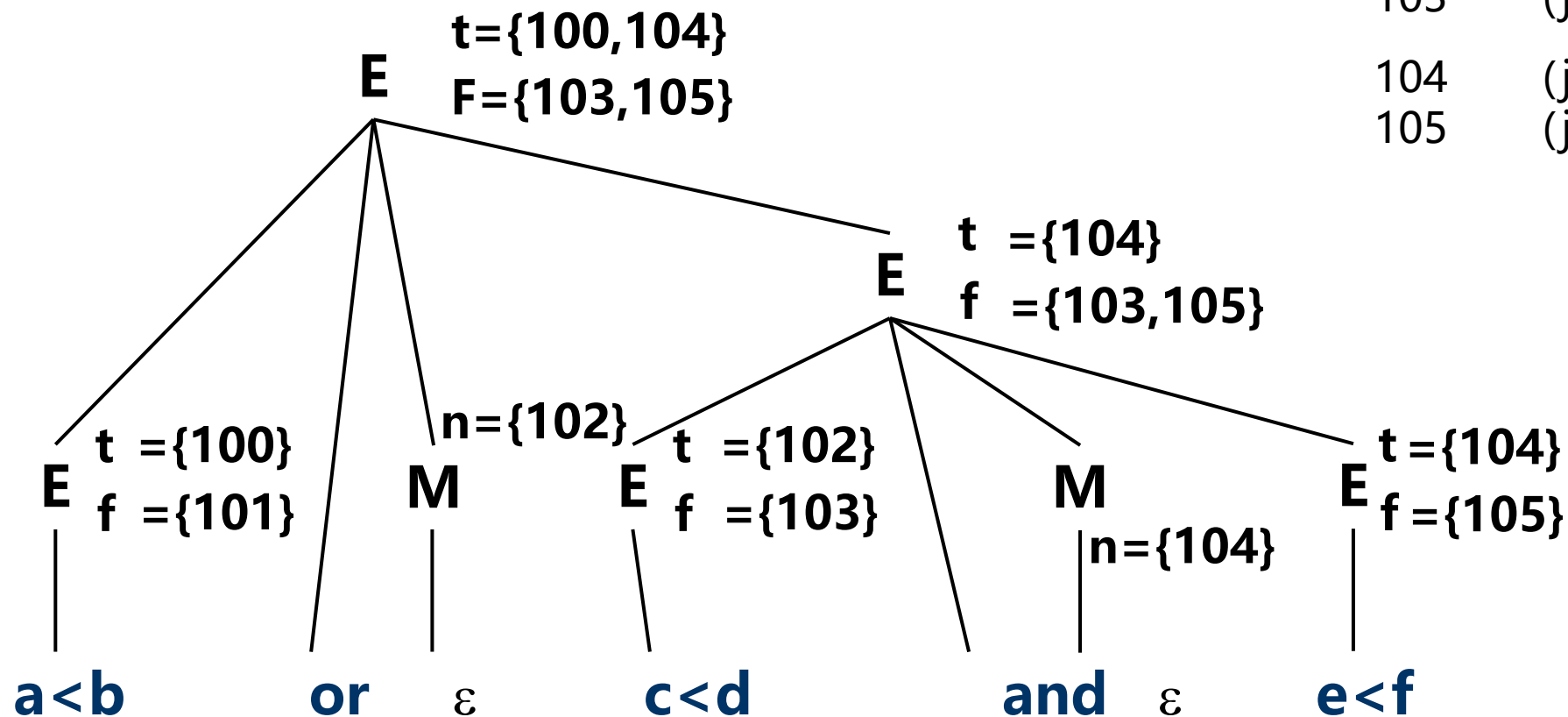
105 (j, -, -, 103)

truelist

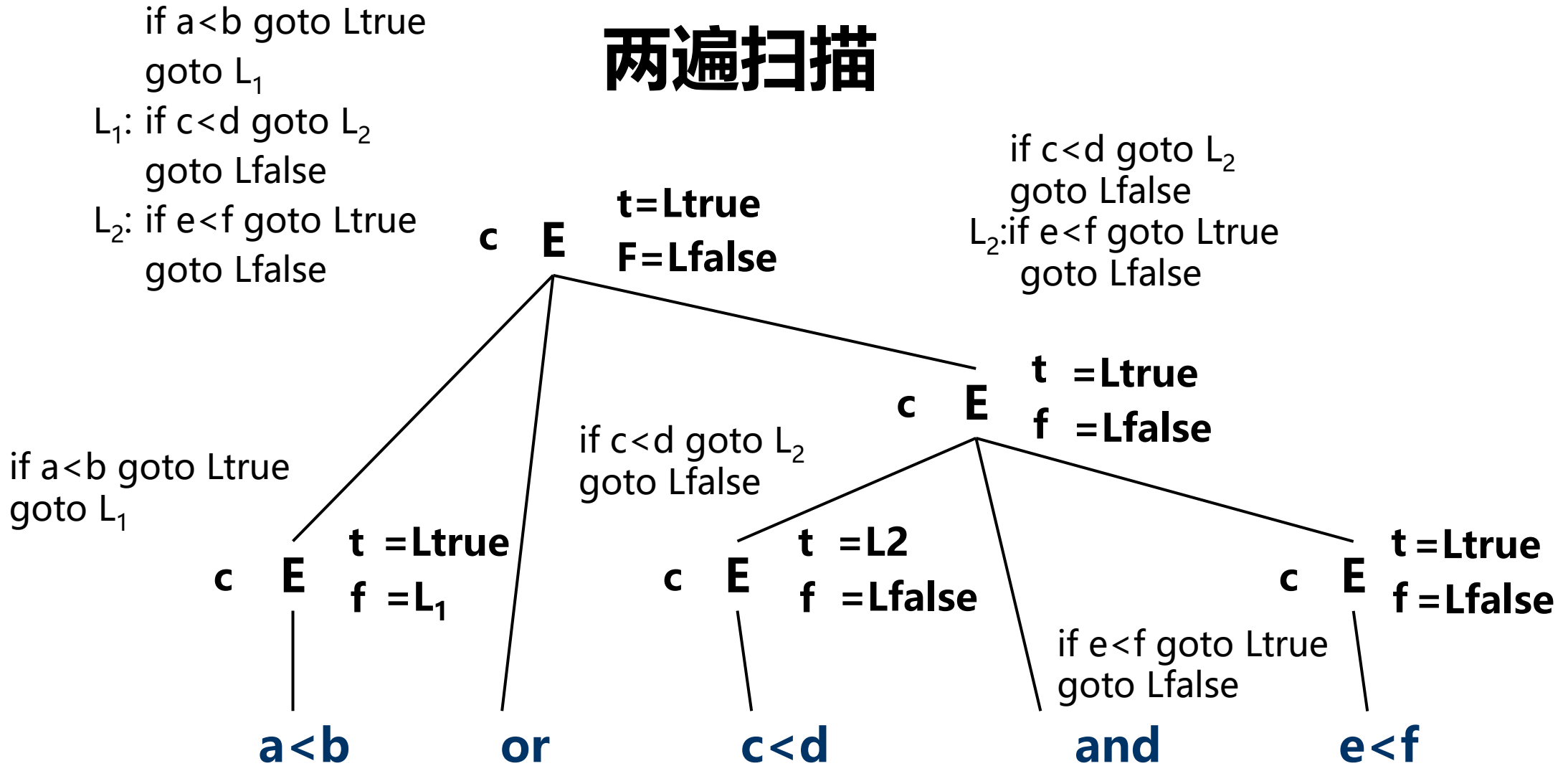
falselist

一遍扫描

100	(j<, a, b, 0)
101	(j, -, -, 102)
102	(j<, c, d, 104)
103	(j, -, -, 0)
104	(j<, e, f, 0)
105	(j, -, -, 0)



两遍扫描



一遍扫描翻译控制流语句

- 考虑下列产生式所定义的语句:

(1) $S \rightarrow \text{if } E \text{ then } S$

(2) $\quad \quad \quad | \text{if } E \text{ then } S \text{ else } S$

(3) $\quad \quad \quad | \text{while } E \text{ do } S$

(4) $\quad \quad \quad | \text{begin } L \text{ end}$

(5) $\quad \quad \quad | A$

(6) $L \rightarrow L; S$

(7) $\quad \quad \quad | S$

- S 表示语句, L 表示语句表,
- A 为赋值语句, E 为一个布尔表达式

if 语句的翻译

相关产生式

$$S \rightarrow \text{if } E \text{ then } S^{(1)} \\ | \text{if } E \text{ then } S^{(1)} \text{ else } S^{(2)}$$

改写后产生式

$$S \rightarrow \text{if } E \text{ then } M S_1$$
$$S \rightarrow \text{if } E \text{ then } M_1 S_1 N \text{ else } M_2 S_2$$
$$M \rightarrow \varepsilon$$
$$N \rightarrow \varepsilon$$

翻译模式

1. $S \rightarrow \text{if } E \text{ then } M \ S_1$
 { backpatch(E.truelist, M.quad);
 S.nextlist:=merge(E.falselist, S_1 .nextlist) }
2. $S \rightarrow \text{if } E \text{ then } M_1 \ S_1 \ N \ \text{else } M_2 \ S_2$
 { backpatch(E.truelist, M_1 .quad);
 backpatch(E.falselist, M_2 .quad);
 S.nextlist:=merge(S_1 .nextlist, N.nextlist, S_2 .nextlist) }
3. $M \rightarrow \varepsilon$ { M.quad:=nextquad }
4. $N \rightarrow \varepsilon$ { N.nextlist:=makelist(nextquad);
 emit('j, - , - , - ') }

示例

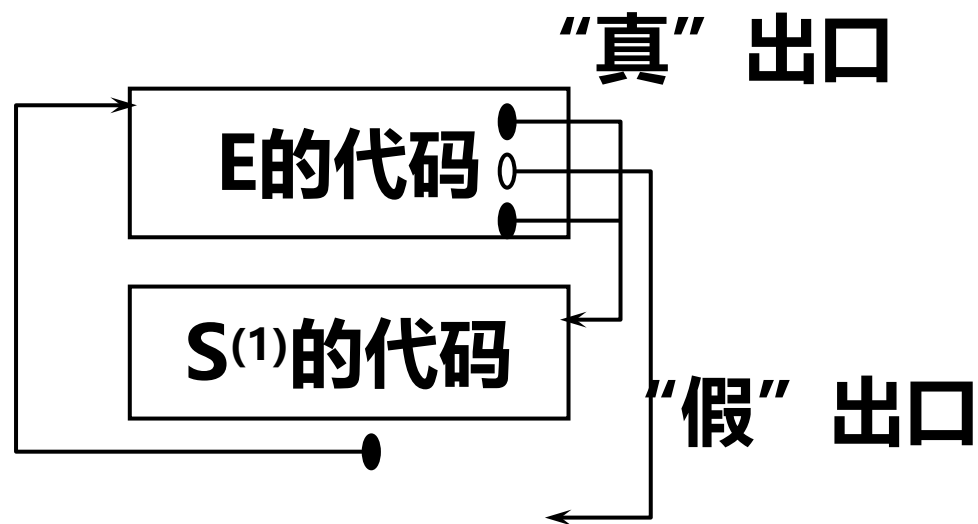
if $a > c$ or $b < d$ then $x := y + z$ else $x := y - z$

```
100      (j>, a, c, 104)
101      (j, -, -, 102)
102      (j<, b, d, 104)
103      (j, -, -, 107)
104      (+, y, z, T1)
105      (:=, T1, -, x)
106      (j, -, -, 0)
107      (-, y, z, T2)
108      (:=, T2, -, x)
S.nextlist:={106}
```

while 语句的翻译

相关产生式

$S \rightarrow \text{while } E \text{ do } S^{(1)}$



为了便于"回填", 改写产生式为:

$S \rightarrow \text{while } M1 \ E \text{ do } M2 \ S1$

$M \rightarrow \epsilon$

1. $S \rightarrow \text{while } M_1 \text{ E do } M_2 S_1$
 {backpatch($S_1.\text{nextlist}$, $M_1.\text{quad}$);
 backpatch($E.\text{truelist}$, $M_2.\text{quad}$);
 $S.\text{nextlist} := E.\text{falselist}$
 emit('j, -, -, ' $M_1.\text{quad}$) }
2. $M \rightarrow \varepsilon$ { $M.\text{quad} := \text{nextquad}$ }

语句 $L \rightarrow L; S$ 的翻译

产生式

$L \rightarrow L; S$

改写为:

$L \rightarrow L_1; M S$

$M \rightarrow \varepsilon$

翻译模式:

1. $L \rightarrow L_1; M S$ { backpatch($L_1.nextlist$, $M.quad$);
 $L.nextlist := S.nextlist$ }
2. $M \rightarrow \varepsilon$ { $M.quad := nextquad$ }

其它几个语句的翻译

$S \rightarrow \text{begin } L \text{ end}$
 $\{ S.\text{nextlist} := L.\text{nextlist} \}$

$S \rightarrow A$
 $\{ S.\text{nextlist} := \text{makelist}() \}$

$L \rightarrow S$
 $\{ L.\text{nextlist} := S.\text{nextlist} \}$

翻译语句

**while (a<b) do
if (c<d) then x:=y+z;**

P195

$S \rightarrow \text{if } E \text{ then } M \ S_1$

{ backpatch(E.truelist, M.quad);

$S.\text{nextlist} := \text{merge}(E.\text{falselist}, S_1.\text{nextlist})$ }

$M \rightarrow_\epsilon \{ M.\text{quad} := \text{nextquad} \}$

$S \rightarrow A \{ S.\text{nextlist} := \text{makelist}() \}$

P195

$S \rightarrow \text{while } M_1 \ E \text{ do } M_2 \ S_1$

{ backpatch($S_1.\text{nextlist}$, $M_1.\text{quad}$);

 backpatch(E.truelist, $M_2.\text{quad}$);

$S.\text{nextlist} := E.\text{falselist}$

 emit('j, - , - , ' $M_1.\text{quad}$) }

$M \rightarrow_\epsilon \{ M.\text{quad} := \text{nextquad} \}$

翻译语句

```
while (a<b) do  
    if (c<d) then x:=y+z;
```

100 (j<, a, b, 102)

101 (j, -, -, 0)

102 (j<, c, d, 104)

103 (j, -, -, 100)

104 (+, y, z, T)

105 (:=, T, -, x)

106 (j, -, -, 100)

107

标号与goto语句

- 标号定义形式: `L: S;`

当这种语句被处理之后，标号L称为“定义了”的。即在符号表中，标号L的“地址”栏将登记上语句S的第一个四元式的地址。

- 标号引用: `goto L;`

向后转移	
L1:	
	
	goto L1;

向前转移	
	goto L1;
	
L1:	

符号表信息

名字	类型	...	定义否	地址
...	...	...	...	...
L	标号		未	r

(P) (j, -, -, 0) ←
...
(q) (j, -, -, p) ←
...
(r) (j, -, -, q) ←

■ 产生式 $S' \rightarrow \text{goto } L$ 的语义动作:

{ 查找符号表;

IF L 在符号表中且"定义否"栏为"已"

THEN GEN(J , -, -, P)

ELSE IF L 不在符号表中

THEN BEGIN

 把 L 填入表中;

 置"定义否"为"未", "地址"栏为nextquad ;

 GEN(J , -, -, 0) END

ELSE BEGIN /* L 在符号表中且 "定义否" 栏为 "未" */

$Q := L$ 的地址栏中的编号;

 置地址栏编号为nextquad ;

 GEN(J , -, -, Q)

END

}

■ 带标号语句的产生式:

$S \rightarrow \text{label } S$

$\text{label} \rightarrow i:$

■ $\text{label} \rightarrow i:$ 对应的语义动作:

1. 若*i*所指的标识符(假定为*L*)不在符号表中, 则把它填入, 置"类型"为"标号", 定义否为"已", "地址"为nextquad ;
2. 若*L*已在符号表中但"类型"不为标号或"定义否"为"已", 则报告出错;
3. 若*L*已在符号表中, 则把标号"未"改为"已", 然后, 把地址栏中的链头(记为*q*)取出, 同时把nextquad填在其中, 最后, 执行BACKPATCH(*q*, nextquad)。

内容线索

√. 中间语言

√. 说明语句的翻译

√. 赋值语句的翻译

√. 布尔表达式的翻译

√. 两遍扫描条件控制及其语句的翻译

√. 一遍扫描条件控制及其语句的翻译

7. 过程调用的处理

过程调用的处理

- **过程调用主要对应两件事:**

- **传递参数**
- **转子 (过程)**

- **传地址:把实在参数的地址传递给相应的形式参数**

- **调用段预先把实在参数的地址传递到被调用段可以拿到的地方;**
- **程序控制转入被调用段之后, 被调用段首先把实在参数的地址抄进自己相应的形式单元中;**
- **过程体对形式参数的引用与赋值被处理成对形式单元的间接访问。**

过程调用的文法

- **过程调用文法:**

(1) $S \rightarrow \text{call id (Elist)}$

(2) $\text{Elist} \rightarrow \text{Elist}, E$

(3) $\text{Elist} \rightarrow E$

- **参数的地址存放在一个队列中**

- **最后对队列中的每一项生成一条par语句**

过程调用的翻译

- 翻译方法：把实参的地址逐一放在转子指令的前面。

例如， `CALL S(A, X+Y)` 翻译为：

中间代码：计算 $X+Y$ ，置于 T 中

`par A    /*第一个参数的地址*/`

`par T    /*第二个参数的地址*/`

`call S   /*转子*/`

3. $Elist \rightarrow E$

{ 初始化queue仅包含E.place }

2. $Elist \rightarrow Elist, E$

{ 将E.place加入到queue的队尾 }

1. $S \rightarrow \text{call id} (Elist)$

```
{ for 队列queue中的每一项p do  
    emit( 'param' p);  
    emit( 'call' id.place) }
```

作业题

■ P217

➤ 1 (前两行, 共4小题)

➤ 3

➤ 4

➤ 6

➤ 7